

Interpretability for Complex Prediction Model - SHAP Values

Jingrui Mu

Department of Mathematics and Statistics, Faculty of Science
McGill University

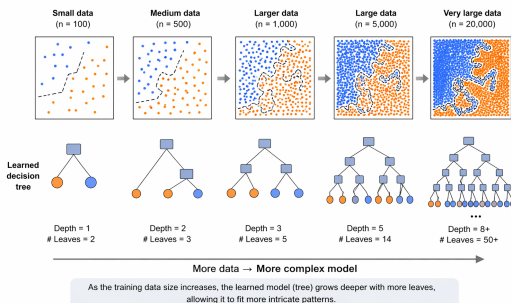
April 23, 2026

Presentation Overview

- 1 Introduction
 - Background
- 2 Methods
 - Shapley Values
 - LIME
 - SHAP
- 3 Simulation
 - Linear-based
 - Nonlinear-based
- 4 Discussion

Model Interpretability and Model Explainability

Model Complexity Grows as Data Grows



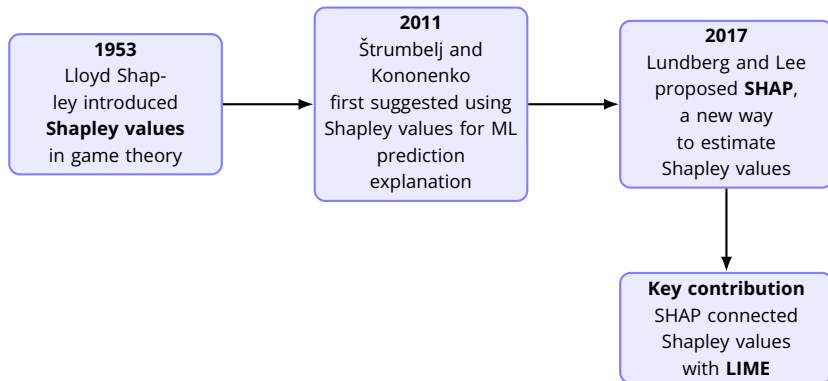
Interpretability

A method is **interpretable** if a user can correctly and efficiently predict the method's results. Kim, Khanna, and Koyejo (2016) [1]

Explainability

Explain model predictions on individual predicted instances

A Brief History of Shapley Values and SHAP



Shapley Values

Background

The idea of SHAP comes from cooperative game theory. In this setting, we consider a set of players $N = \{1, \dots, p\}$. For every subset $S \subseteq N$, a value function $v(S)$ is defined to represent the total payoff that the players in S can achieve by cooperating. The quantity $v(\emptyset)$ is taken as the baseline payoff, namely the value obtained when no player participates.

Shapley Values

Example

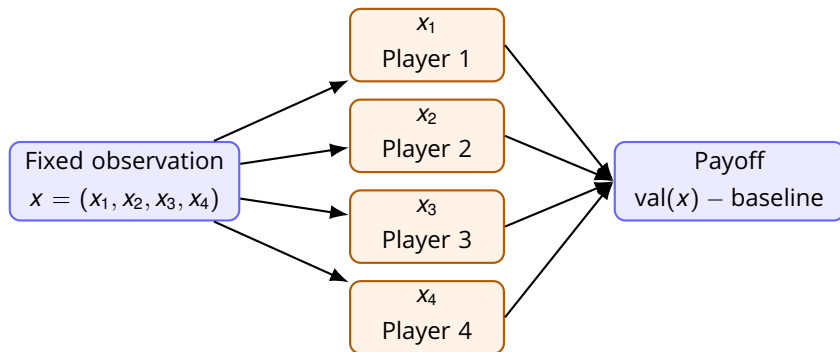
Consider of three players $N = \{A, B, C\}$ and then we possibly can have the following profits in different subsets: $\text{val}(\emptyset) = 0$, $\text{val}(\{A\}) = 1$, $\text{val}(\{B\}) = 2$, $\text{val}(\{C\}) = 0$, $\text{val}(\{A, B\}) = 10$, $\text{val}(\{A, C\}) = 1$, $\text{val}(\{B, C\}) = 2$, $\text{val}(\{A, B, C\}) = 10$. For player A , the marginal contribution depends on the subset to which A is added. For example, $v(\{A\}) - v(\emptyset) = 1$, while $\text{val}(\{A, B\}) - \text{val}(\{B\}) = 8$. Thus, the contribution of player A is not a single fixed quantity; it varies with the subset under consideration.

Shapley Values

What does the Shapley value resolve ?

The Shapley value provides a fair and mathematically justified allocation of the total payoff by averaging the marginal contribution of a player over all possible subsets.

Shapley Values as a Cooperative Game



Shapley value of feature j :

average marginal contribution of feature j over all possible subsets of features

Apartment Price Example and Shapley Value

Prediction decomposition

$$400,000 = 380,000 + 30,000 \\ + 10,000 - 10,000 - 10,000$$

- **Top school nearby:**

+\$30,000

- **Park nearby:** +\$10,000

- **Floor: 2nd:** -\$10,000

- **Area: 50 m^2 :** -\$10,000

Baseline prediction: \$380,000

Shapley value of top-school-nearby

Consider all subsets excluding top-school-nearby:

$\emptyset$, {park}, {area}, {floor},
 {park, area}, {park, floor}, {area, floor},
 {park, area, floor}

For each subset S , compute

$$\text{val}(S \cup \{\text{top-school}\}) - \text{val}(S).$$

Then average across all such subsets.

Shapley_{top-school} = average marginal contribution
 over all coalitions.

Shapley Value Formula

Main idea: The Shapley value of feature j measures its average contribution to the prediction over all possible feature coalitions.

$$\phi_j(val_{\mathbf{x}_i}) = \sum_{S \subseteq \{1, \dots, p\} \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} (val_{\mathbf{x}_i}(S \cup \{j\}) - val_{\mathbf{x}_i}(S)) \quad (1)$$

- S : a subset of features not containing feature j
- p : total number of features
- $\mathbf{x}_i$: the feature vector of the instance being explained
- $val_{\mathbf{x}_i}(S)$: the model prediction using the features in S , marginalized over the remaining features

For each subset S , we compute the marginal contribution of feature j :

$$val_{\mathbf{x}_i}(S \cup \{j\}) - val_{\mathbf{x}_i}(S),$$

and then average these contributions using the Shapley weights.

Practical Approximation of the Value Function

Value function:

$$val_{x_i}(S) = \int \hat{f}(x_S^{(i)}, x_C) dP_{X_C}(x_C) - \mathbb{E}[\hat{f}(X)] \quad (2)$$

Notation:

- $x_S^{(i)}$: observed feature values of instance i , restricted to subset S
- X_C : remaining features, treated as random

Practical approximation: In practice, the integral is approximated by averaging over M background samples, where M background samples are sampled from the dataset:

$$\int \hat{f}(x_S^{(i)}, x_C) dP_{X_C}(x_C) \approx \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_C^{(m)}) \quad (3)$$

Approximation of the Marginal Contribution

Using the background samples, we approximate

$$val_{\mathbf{x}_i}(S) \approx \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_C^{(m)}) - \frac{1}{M} \sum_{m=1}^M \hat{f}(\mathbf{x}^{(m)}) \quad (4)$$

and

$$val_{\mathbf{x}_i}(S \cup \{j\}) \approx \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_j^{(i)}, x_{C/j}^{(m)}) - \frac{1}{M} \sum_{m=1}^M \hat{f}(\mathbf{x}^{(m)}) . \quad (5)$$

Therefore, the marginal contribution of feature j is

$$val_{\mathbf{x}_i}(S \cup \{j\}) - val_{\mathbf{x}_i}(S) = \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_j^{(i)}, x_{C/j}^{(m)}) - \frac{1}{M} \sum_{m=1}^M \hat{f}(x_S^{(i)}, x_C^{(m)}) . \quad (6)$$

Properties of Shapley Values

Shapley values are uniquely characterized by four key properties:

- 1 **Efficiency** The feature contributions add up to the prediction difference from the baseline:

$$\sum_{j=1}^p \phi_j(\mathbf{x}_i) = \hat{f}(\mathbf{x}_i) - \mathbb{E}[\hat{f}(X)].$$

- 2 **Symmetry** If two features contribute equally to every possible subset, then they receive the same Shapley value.
- 3 **Dummy** If a feature never changes the prediction when added to any subset, then its Shapley value is 0.
- 4 **Additivity** For two value functions $val_{\mathbf{x}_i}$ and $val_{\mathbf{x}_i}^+$, the Shapley value for the sum equals the sum of the Shapley values:

$$\phi_j^{val+val^+}(\mathbf{x}_i) = \phi_j(\mathbf{x}_i) + \phi_j^+(\mathbf{x}_i).$$

How to Obtain Shapley Values?

Main challenge: Computing the exact Shapley value requires evaluating all possible feature subsets, which becomes computationally expensive as the number of features grows.

Monte Carlo approximation (Štrumbelj and Kononenko, 2014):

$$\hat{\phi}_j(val_{\mathbf{x}_i}) = \frac{1}{B} \sum_{b=1}^B \left[\hat{f}(\mathbf{x}_{+j}^{(b)}) - \hat{f}(\mathbf{x}_{-j}^{(b)}) \right] \quad (7)$$

For each repetition b :

- 1 Generate a random permutation of the features.
- 2 Draw a random background observation from the data.
- 3 Construct two hybrid vectors: $\mathbf{x}_{+j}^{(b)}$ (feature j is included) and $\mathbf{x}_{-j}^{(b)}$ (feature j is excluded)

Monte Carlo Approximation of Shapley Values

Idea: Approximate the Shapley value by averaging the marginal contribution of feature j over many random permutations and background samples.

Algorithm 1 Monte Carlo approximation of the Shapley value for feature j at instance $\mathbf{x}_j$

- 1: **Input:** Number of resamples B , instance of interest $\mathbf{x}_j$, feature index j , data matrix X , and fitted prediction function $\hat{f}$
 - 2: **Output:** Approximate Shapley value $\phi_j(\mathbf{x}_j)$
 - 3: **for** $b = 1, \dots, B$ **do**
 - 4: Draw a random background observation $\mathbf{x}^{(b)}$ from X
 - 5: Draw a random permutation $\mathbf{o} = ((1), \dots, (p))$ of the feature indices and
 Let $S^{(b)} = \{(1), \dots, (j-1)\}$ denote the set of features that appear before feature j in the permutation
 - 6: Construct these two input vectors $\mathbf{x}_{+j}^{(b)} = (x_{(1)}, \dots, x_{(j-1)}, x_{(j)}, x_{(j+1)}^{(b)}, \dots, x_{(p)}^{(b)})$ and $\mathbf{x}_{-j}^{(b)} = (x_{(1)}, \dots, x_{(j-1)}, x_{(j)}^{(b)}, x_{(j+1)}^{(b)}, \dots, x_{(p)}^{(b)})$
 - 7: Compute the sampled marginal contribution $\phi_j^{(b)}(\mathbf{x}_j) = \hat{f}(\mathbf{x}_{+j}^{(b)}) - \hat{f}(\mathbf{x}_{-j}^{(b)})$
 - 8: **end for**
 - 9: Compute the Monte Carlo estimate of the Shapley value $\phi_j(\mathbf{x}_j) = \frac{1}{B} \sum_{b=1}^B \phi_j^{(b)}(\mathbf{x}_j)$
-

Limitations of Shapley Values

Main limitation: Computing Shapley values can be computationally expensive.

- **Exponential complexity:** Exact computation requires evaluating all possible feature subsets, i.e., 2^p subsets.
- **Simulation of feature absence:** The absence of a feature is usually approximated by replacing it with values from background observations, which introduces additional sampling variability.
- **Approximation trade-off:** In practice, the exponential number of coalitions is handled by Monte Carlo sampling and a limited number of resamples B .

Shapley values are theoretically appealing, but their practical use often requires approximation.

Illustration of the Intuition Behind LIME

Ribeiro, Singh, and Guestrin (2016) used the following figure to illustrate the intuition behind LIME.

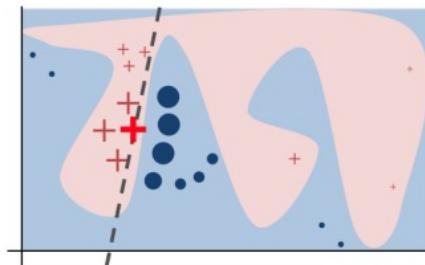


Figure: Toy example illustrating LIME

LIME and Its Connection to Shapley Values

For an instance $\mathbf{x}_i$, LIME explains the black-box prediction $\hat{f}(\mathbf{x}_i)$ by fitting a simple interpretable model g locally around $\mathbf{x}_i$.

$$\text{explanation}(\mathbf{x}_i) = \arg \min_{g \in G} L(\hat{f}, g, \pi_{\mathbf{x}_i}) + \Omega(g) \quad (8)$$

- L : how well g matches $\hat{f}$ locally
- $\pi_{\mathbf{x}_i}$: neighborhood weight around $\mathbf{x}_i$
- $\Omega(g)$: penalty for model complexity

LIME provides a local surrogate explanation, while SHAP can be viewed as connecting local explanation to the Shapley-value principle of fair feature attribution.

How Does LIME Sample Local Points?

Main idea: LIME learns the local behavior of $\hat{f}$ around the instance $\mathbf{x}_i$ by generating perturbed samples near $\mathbf{x}_i$ and weighting them by proximity.

- **Tabular data:** perturb each feature using the training-data distribution
 - continuous features: sample around the empirical mean and standard deviation
 - categorical features: sample according to empirical category frequencies
- **Text data:** represent the text as a binary word-presence vector, then randomly remove words to create neighboring samples
- **Image data:** represent the image by superpixels, then randomly switch superpixels on or off to create neighboring samples

LIME creates simple perturbations around the instance of interest and fits a weighted local surrogate model on these samples.

Sparse Linear Explanations Using LIME

Idea: After generating perturbed samples around $\mathbf{x}_i$, LIME fits a sparse linear model to approximate the black-box prediction locally.

Algorithm 2 Sparse linear explanation using LIME

- 1: **Input:** Number of samples N , instance $\mathbf{x}_i$, similarity kernel $\pi_{\mathbf{x}_i}$, black-box model $\hat{f}$, K is chosen by the user and controls the sparsity, i.e., the number of features shown in the explanation.
- 2: **Output:** Sparse local explanation model $\hat{g}$
- 3: **for** $n = 1, \dots, N$ **do**
- 4: Sample a perturbed point $\mathbf{x}_i'^{(n)}$ around $\mathbf{x}_i$
- 5: Compute its weight $\pi_{\mathbf{x}_i}(\mathbf{x}_i'^{(n)})$
- 6: Evaluate the black-box prediction $\hat{f}(\mathbf{x}_i'^{(n)})$
- 7: **end for**
- 8: Fit the sparse linear model

$$\hat{g} = \arg \min_{g \in \mathcal{G}} L(\hat{f}, g, \pi_{\mathbf{x}_i}) + \Omega(g)$$

Model class:

$$g(\mathbf{x}') = \mathbf{w}_g^\top \mathbf{x}', \quad \pi_{\mathbf{x}_i}(\mathbf{x}') = \exp\left(-\frac{D(\mathbf{x}_i, \mathbf{x}')^2}{\sigma^2}\right), \quad \Omega(g) = \mathbb{I}(\|\mathbf{w}_g\|_0 > K)$$

Limitations of LIME

- **Neighborhood choice is difficult:** Defining an appropriate local neighborhood remains an open problem, especially for tabular data.
- **Instability across nearby instances:** Very similar observations can receive quite different explanations, as shown by Alvarez-Melis and Jaakkola (2018).
- **Sampling variability:** Repeating the sampling procedure may produce different local surrogate models and therefore different explanations.

LIME is intuitive and flexible, but its explanations can depend strongly on how the local neighborhood is sampled.

SHAP and Its Connection to LIME

By the efficiency property, a Shapley explanation is an **additive feature attribution model**:

$$\hat{f}(\mathbf{x}_i) = \mathbb{E}[\hat{f}(X)] + \sum_{j=1}^p \phi_j.$$

SHAP writes the explanation model as

$$g(\mathbf{x}'_i) = \phi_0 + \sum_{j=1}^p \phi_j x'_{ij}.$$

- ϕ_0 : baseline prediction, ϕ_j : contribution of feature j
- $\mathbf{x}'_i \in \{0, 1\}^p$: simplified binary feature vector. $x'_{ij} = 1$ means feature j is present; $x'_{ij} = 0$ means it is absent

Both LIME and SHAP use additive local explanations, but SHAP grounds the explanation in the axiomatic framework of Shapley values.

Additional Properties of SHAP



Local Accuracy

$$\hat{f}(\mathbf{x}_i) = g(\mathbf{x}'_i) = \phi_0 + \sum_{j=1}^p \phi_j x'_{ij}.$$



Missingness

$$x'_{ij} = 0 \implies \phi_j = 0.$$

Additional Properties of SHAP



Consistency Let

$$\hat{f}_{\mathbf{x}}(\mathbf{x}') = \hat{f}(h_{\mathbf{x}}(\mathbf{x}')).$$

If for all $\mathbf{x}' \in \{0, 1\}^p$,

$$\hat{f}'_{\mathbf{x}}(\mathbf{x}') - \hat{f}'_{\mathbf{x}}(\mathbf{x}'_{-j}) \geq \hat{f}_{\mathbf{x}}(\mathbf{x}') - \hat{f}_{\mathbf{x}}(\mathbf{x}'_{-j}),$$

then

$$\phi_j(\hat{f}', \mathbf{x}) \geq \phi_j(\hat{f}, \mathbf{x}).$$

if feature j contributes more in the new model for every subset, then its attribution should not decrease.

Unique SHAP Solution

Lundberg and Lee (2017) showed that there is a unique additive explanation model satisfying **local accuracy**, **missingness**, and **consistency**.

$$\phi_j(f, \mathbf{x}_i) = \sum_{\mathbf{x}'_i} \frac{|\mathbf{x}'_i|! (p - |\mathbf{x}'_i| - 1)!}{p!} \left[f_{\mathbf{x}}(\mathbf{x}'_i) - f_{\mathbf{x}}(\mathbf{x}'_i \setminus j) \right]$$

- $\mathbf{x}'_i \in \{0, 1\}^p$: simplified input
- $|\mathbf{x}'_i|$: number of included features
- $\mathbf{x}'_i \setminus j$: input with feature j absent

KernelSHAP (Linear LIME + SHAP)

Under the additive explanation model g , KernelSHAP chooses

$$\Omega(g) = 0, \pi_{\mathbf{x}_i}(\mathbf{x}'_i) = \frac{p - 1}{\binom{p}{|\mathbf{x}'_i|} |\mathbf{x}'_i| (p - |\mathbf{x}'_i|)},$$

and

$$L(\hat{f}, g, \pi_{\mathbf{x}_i}) = \sum_{\mathbf{x}'_i} \left[\hat{f}(h_{\mathbf{x}_i}^{-1}(\mathbf{x}'_i)) - g(\mathbf{x}'_i) \right]^2 \pi_{\mathbf{x}_i}(\mathbf{x}'_i).$$

- $|\mathbf{x}'_i|$: number of present features in the simplified input
- $\pi_{\mathbf{x}_i}(\mathbf{x}'_i)$: Shapley kernel weight
- L : weighted local least-squares loss
- $\Omega(g) = 0$: no additional complexity penalty

When $|\mathbf{x}'_i| \in \{0, p\}$, the kernel weight becomes infinite, forcing the explanation to match the baseline and the full prediction exactly.

KernelSHAP Receptes

Algorithm 3 Kernel SHAP for one observation x

- 1: **Input:** fitted model f , instance x , background dataset $\{x^{(1)}, \dots, x^{(M)}\}$, number of sampled coalitions K
- 2: **Output:** estimated SHAP values $\hat{\phi}_1, \dots, \hat{\phi}_p$
- 3: **for** $k = 1, \dots, K$ **do**
- 4: Sample a coalition vector $x'^{(k)} \in \{0, 1\}^p$
- 5: **for** $m = 1, \dots, M$ **do**
- 6: Construct a hybrid input $\tilde{x}^{(k,m)}$ by setting and evaluate $f(\tilde{x}^{(k,m)})$

$$\tilde{x}_j^{(k,m)} = \begin{cases} x_j, & z_j'^{(k)} = 1, \\ x_j^{(m)}, & z_j'^{(k)} = 0, \end{cases} \quad j = 1, \dots, p$$

- 7: **end for**
 - 8: Compute $y^{(k)} = \frac{1}{M} \sum_{m=1}^M f(\tilde{x}^{(k,m)})$ and let $s_k = |x'^{(k)}|$ and define $\pi_{\mathbf{x}}^{(k)} = \frac{p-1}{\binom{p}{s_k} s_k (p-s_k)}$
 - 9: **end for**
 - 10: Fit the weighted least-squares problem $\min_{\phi_0, \phi_1, \dots, \phi_p} \sum_{k=1}^K \pi_{\mathbf{x}}^{(k)} \left(y^{(k)} - \phi_0 - \sum_{j=1}^p \phi_j x_j'^{(k)} \right)^2$
 - 11: Return $\hat{\phi}_1, \dots, \hat{\phi}_p$
-

Simulation Design: Linear Gaussian Model

- **Sample size and replications** $n = 500$, $p = 5$, $n_{\text{reps}} = 100$

- **Covariate generation**

$$X_i \sim N_p(\mu, \Sigma), \quad i = 1, \dots, n,$$

where

$$\mu = (0, 0, 0, 0, 0)^\top, \quad \Sigma_{jk} = \rho^{|j-k|}, \quad j, k = 1, \dots, p.$$

- **Fixed design**

- The design matrix $X \in \mathbb{R}^{500 \times 5}$ is generated once.
- The same X is used across all replications.

- **Outcome model** $Y_i = \beta_0 + X_i^\top \beta + \varepsilon_i$, with $\beta_0 = 1.5$, $\beta = (0.5, 2.0, -1.0, 1.5, -1.0)^\top$.

- **Noise distribution**

$$\varepsilon_i \sim N(0, \sigma_\varepsilon^2), \quad \sigma_\varepsilon = 1.$$

- **Replication scheme**

- Across the 100 replications, only the noise vector ε is regenerated.
- The predictor matrix X remains fixed.

Beeswarm Plot ($p = 5$, $n = 500$)

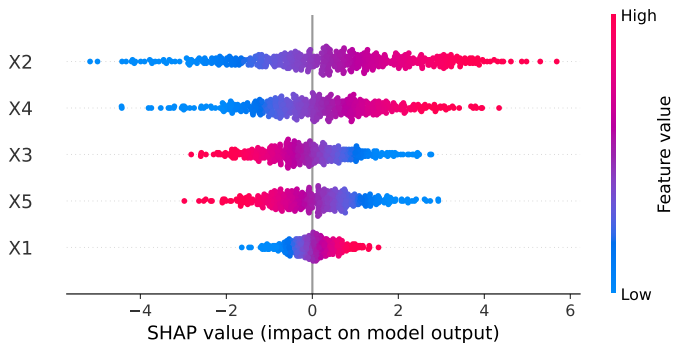


Figure: Beeswarm Plot ($n = 500$, $p = 5$)

Beeswarm Plot ($p = 500, n = 500$)

Sparse Linear Model: $Y_i = \beta_0 + X_i^\top \beta + \varepsilon_i$, with $\beta_0 = 1.5$, $\beta = (0.5, 2.0, -1.0, 1.5, -1.0, 0, \dots, 0)^\top$.

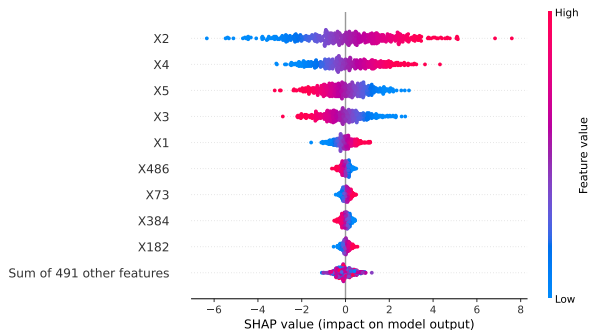


Figure: Beeswarm Plot ($n = 500, p = 500$)

Simulation Design: GAM Model

- **Sample size and replications** $n = 500$, $p = 5$, $n_{\text{reps}} = 100$
- **Covariate generation** $X_i \sim N_p(\mu, \Sigma)$, $i = 1, \dots, n$, where

$$\mu = (0, 0, 0, 0, 0)^\top, \quad \Sigma_{jk} = \rho^{|j-k|}, \quad j, k = 1, \dots, p.$$

- **Fixed design**
 - The design matrix $X \in \mathbb{R}^{500 \times 5}$ is generated once.
 - The same X is used across all replications.

- **Outcome model**

$$Y_i = \beta_0 + f_1(X_{i1}) + f_2(X_{i2}) + f_3(X_{i3}) + \varepsilon_i,$$

with intercept $\beta_0 = 1.5$, and component functions

$$f_1(x) = 2\log(|x|), \quad f_2(x) = 1.5\sqrt{x}, \quad f_3(x) = -2\log(|x|).$$

The remaining covariates X_4 and X_5 are inactive.

- **Noise distribution**

$$\varepsilon_i \sim N(0, \sigma_\varepsilon^2), \quad \sigma_\varepsilon = 1.$$

Beeswarm Plot ($p = 5$, $n = 500$)

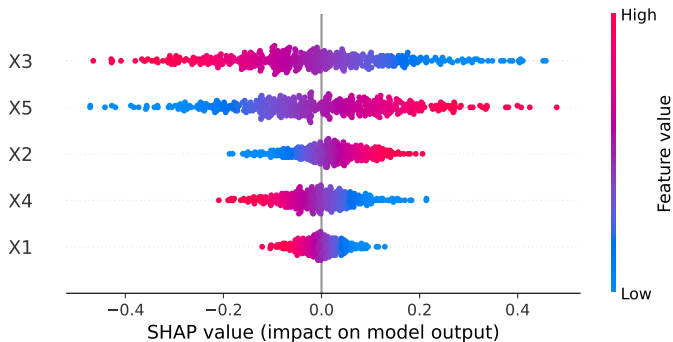


Figure: Beeswarm Plot (fitted under Linear regression)

Beeswarm Plot ($p = 5$, $n = 500$)

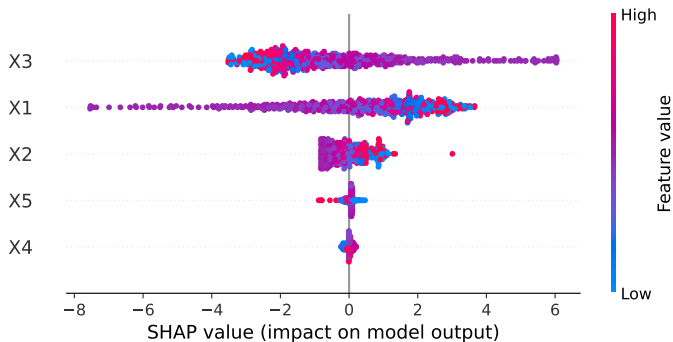


Figure: Beeswarm Plot (fitted under GAM regression)

Statistical Inference for SHAP

People often use SHAP summaries such as mean absolute SHAP to rank variables, but how can we do valid statistical inference for those quantities?

Statistical Inference for SHAP

- Let the true regression function be

$$\mu_0(x) = \mathbb{E}[Y \mid X = x].$$

- For feature j , the paper defines the **SHAP curve**

$$\psi_j(x; \mu_0) = \frac{1}{p} \sum_{S \subseteq [p] \setminus \{j\}} \binom{p-1}{|S|}^{-1} \left\{ \nu_x(S \cup \{j\}; \mu_0) - \nu_x(S; \mu_0) \right\},$$

where

$$\nu_x(S; \mu_0) = \mathbb{E}[\mu_0(x_S, X_{-S})].$$

- The main global target is the d -th moment of the SHAP curve:

$$\theta_{j,d} = \mathbb{E}[\psi_j(X; \mu_0)^d].$$

This includes: $d = 1$: mean absolute SHAP, and $d = 2$: mean squared SHAP.

Statistical Inference for SHAP

- $\psi_j(\cdot; \mu_0)$ depends on the unknown function μ_0 .
- A naive plug-in estimator

$$\hat{\theta}_{j,d}^{\text{plug-in}} = \frac{1}{n} \sum_{i=1}^n |\hat{\psi}_j(X_i)|^d$$

is generally too biased for valid asymptotic inference.

Statistical Inference of SHAP

- For $d \geq 2$, the paper introduces the de-biased / Neyman-orthogonal score

$$m_d(Z, Z'; g) = |\phi(X)|^d + \gamma(X)\{\psi_j(X, X'; \mu) - \phi(X)\} + \alpha(X')\{Y - \mu(X)\},$$

where $g = (\phi, \mu, \gamma, \alpha)$, $Z = (X, Y)$, $Z' = (X', Y')$.

- Z and Z' are two independent samples from the data generating distribution.
- $|\phi(X)|^d$: plug-in target,
- $\gamma(X)\{\psi_j(X, X'; \mu) - \phi(X)\}$: de-biasing for the SHAP curve,
- $\alpha(X')\{Y - \mu(X)\}$: de-biasing for the regression nuisance μ_0 .
- They then symmetrize m_d and construct a **de-biased U-statistic** estimator for θ_d .
- For $1 \leq d < 2$, since $|t|^d$ is irregular, they first smooth the target and then apply the same de-biasing strategy to the smoothed version.

Merci!

Q & A